

План демонстрации разработанного программного продукта

Tsunami — CLI и веб-интерфейс для системы нагрузочного тестирования

1. Назначение демонстрации

Цель демонстрации — показать работу разработанного инструмента Tsunami при нагрузочном тестировании HTTP и gRPC сервисов. Основной акцент делается не просто на факте отправки запросов, а на том, что весь инструмент поставляется единым бинарным файлом без внешних зависимостей и включает как CLI-интерфейс, так и браузерный веб-интерфейс с визуализацией метрик в реальном времени.

Во время демонстрации необходимо показать, что разработанный инструмент позволяет:

- запускать HTTP и gRPC нагрузочные тесты из командной строки без установки дополнительных рантаймов;
- управлять тестами через браузерный веб-интерфейс и наблюдать метрики в реальном времени;
- гибко настраивать интенсивность нагрузки в формате N/Vunit (например, 100/1s);
- собирать и отображать перцентили задержки p50, p90, p95, p99;
- интегрировать тестирование в CI/CD-пайплайны через REST API.

Отдельно следует подчеркнуть, что React SPA встроен непосредственно в бинарный файл через `go:embed` — никаких внешних файлов или серверов для фронтенда не требуется.

2. Состав демонстрационного стенда

Для демонстрации используется минимальный тестовый стенд. Он должен быть подготовлен заранее, чтобы во время выступления не тратить время на установку зависимостей.

Компонент	Назначение	Что должно быть подготовлено
Бинарный файл Tsunami	Основной инструмент	Скомпилирован или установлен через Homebrew
HTTP-сервис для тестирования	Цель нагрузочного теста	Запущен httpbin или любой HTTP-сервис
gRPC-сервис для тестирования	Цель gRPC-теста	Запущен тестовый gRPC-сервер
Браузер	Просмотр веб-интерфейса	Открыт и готов к подключению
Терминал	Запуск CLI-команд	Подготовлены команды для копипаста

3. Предварительная подготовка перед демонстрацией

Перед началом демонстрации необходимо выполнить техническую подготовку. Эти действия лучше сделать заранее и во время защиты только кратко показать результат.

1. Проверить наличие бинарного файла `tsunami` (или установку через `brew install paniccaaa/tap/tsunami`).
2. Запустить HTTP-сервис-цель (например, `docker run -p 80:80 kennethreitz/httpbin`).
3. Запустить gRPC-сервис-цель для демонстрации режима `grpc`.

4. Открыть браузер и убедиться в доступности `http://localhost:8080`.
5. Подготовить набор команд для демонстрации в терминале.
6. Убедиться, что порт 8080 (или настроенный) свободен.
7. Проверить, что версия Tsunami выводится без ошибок: `tsunami --version`.

4. Требования к целевому HTTP-сервису

Для наглядной демонстрации нагрузки в качестве цели рекомендуется использовать `httpbin` — простой HTTP-сервис, возвращающий предсказуемые ответы. Он позволяет продемонстрировать различные сценарии: успешные запросы, таймауты, коды ошибок.

```
# Запуск httpbin через Docker
docker run -d -p 80:80 kennethreitz/httpbin
```

Проверить доступность:

```
curl http://localhost/get
```

5. Демонстрационный сценарий

Этап 1. Показ установки и версии На первом этапе демонстрируется, что Tsunami — единый бинарный файл без зависимостей. Можно показать способы установки:

```
# Через Homebrew
brew install paniccaaa/tap/tsunami

# Или прямой запуск скачанного бинаря
./tsunami --version
```

Ожидаемый результат: инструмент запускается немедленно без установки JVM, Python, Node.js и иных рантаймов.

Этап 2. HTTP нагрузочный тест через CLI Основной режим использования — команда `attack`. Следует показать запуск теста с понятными параметрами:

```
tsunami attack \
  -u http://localhost/get \
  -r 100/1s \
  -d 15s \
  -w 50
```

Во время показа пояснить параметры: `-r 100/1s` — 100 запросов в секунду; `-d 15s` — длительность теста 15 секунд; `-w 50` — 50 параллельных воркеров.

Ожидаемый результат: в терминале отображается прогресс-индикатор и по завершении — таблица с перцентилями p50, p90, p95, p99, распределением статус-кодов и типами ошибок.

Этап 3. Запуск веб-интерфейса Далее демонстрируется браузерный режим работы:

```
tsunami serve --port 8080
```

После запуска открыть в браузере `http://localhost:8080`. Показать, что: - форма конфигурации теста доступна прямо в браузере; - никаких внешних файлов не потребовалось — SPA встроен в бинарь.

Этап 4. Запуск теста через веб-интерфейс и наблюдение метрик Запустить тест прямо из браузера, заполнив форму: URL цели, интенсивность, длительность. После старта показать:

- графики задержки с перцентилями p50/p90/p95/p99, обновляемые каждые 50 мс;
- распределение статус-кодов;
- счётчики успешных запросов и ошибок в реальном времени.

Подчеркнуть, что метрики транслируются через WebSocket с интервалом 50 мс — без перезагрузки страницы.

Этап 5. Остановка теста и экспорт результатов Показать остановку теста кнопкой в браузере или через REST API:

```
curl -X POST http://localhost:8080/api/attack/stop
```

Затем показать экспорт результатов в JSON:

```
curl http://localhost:8080/api/attack/results/download -o results.json
```

Этап 6. gRPC нагрузочный тест Продемонстрировать поддержку gRPC — протокола, которого нет у большинства лёгких аналогов:

```
tsunami grpc \  
-u localhost:50051 \  
-r 50/1s \  
-d 10s
```

Этап 7. Демонстрация производительности Запустить тест с высокой интенсивностью для демонстрации достижения целевого показателя:

```
tsunami attack \  
-u http://localhost/get \  
-r 1000/1s \  
-d 10s \  
-w 50
```

Ожидаемый результат: инструмент стабильно обеспечивает более 1 000 RPS при 50 параллельных воркерах — нефункциональное требование выполнено.

6. Демонстрация дистрибуции

Кратко показать, что инструмент распространяется для 6 платформ через GitHub Releases и Homebrew. Можно открыть страницу релизов и показать доступные артефакты: `tar.gz` для Linux и macOS, `zip` для Windows.

```
# Установка через Homebrew одной командой  
brew install panicaaa/tap/tsunami
```

7. Критерии успешной демонстрации

Критерий	Ожидаемый результат
Бинарь запускается без зависимостей	<code>tsunami --version</code> выводит версию без ошибок
HTTP-тест выполнен	Таблица с перцентилями p50/p90/p95/p99 выведена
Веб-интерфейс открывается	SPA доступен по <code>localhost:8080</code> без внешних файлов
Метрики обновляются в реальном времени	Графики меняются каждые 50 мс без перезагрузки
gRPC-тест выполнен	Результаты выведены аналогично HTTP-режиму
Производительность $\geq 1\,000$ RPS	Достигается при 50 воркерах
Экспорт результатов работает	JSON-файл с метриками скачан

8. План выступления во время демонстрации

Время	Действие	Комментарий докладчика
0:00–0:30	Описать цель показа	Единый бинарь: CLI + веб-интерфейс без зависимостей
0:30–1:00	Показать установку и <code>--version</code>	Никакого рантайма не требуется
1:00–2:30	Запустить <code>attack</code> и показать вывод	Перцентили, коды ответов, классификация ошибок
2:30–4:00	Запустить <code>serve</code> , открыть браузер	Форма конфигурации, запуск теста, живые графики
4:00–4:30	Остановить тест, скачать JSON	REST API для CI/CD интеграции
4:30–5:00	Показать <code>grpc</code> команду	Поддержка gRPC — отличие от Vegeta и hey
5:00–5:30	Запустить тест на 1 000 RPS	Нефункциональное требование выполнено
5:30–6:00	Показать GitHub Releases	Дистрибуция для 6 платформ, Homebrew

9. Что подготовить на случай технических проблем

Если live-демонстрация не запускается, показать заранее подготовленные выводы команд. Рекомендуется иметь:

- скриншот терминала с выводом `tsunami attack` и таблицей перцентилей;
- скриншот веб-интерфейса с активным тестом и живыми графиками;
- скриншот страницы GitHub Releases с артефактами для 6 платформ;
- заранее скачанный `results.json` с результатами тестирования;
- короткую версию сценария на 3 минуты (только CLI `attack` + `serve`).

10. Чек-лист перед выступлением

- Бинарный файл `tsunami` присутствует и запускается
- HTTP-сервис `httpbin` запущен и отвечает на `curl`
- gRPC-сервис запущен
- Порт 8080 свободен
- Браузер открыт и готов к подключению
- Терминал с подготовленными командами открыт
- Скриншоты на случай технических проблем подготовлены
- Страница GitHub Releases открыта в отдельной вкладке

11. Итоговый тезис для завершения демонстрации

Разработанный инструмент `Tsunami` решает практическую задачу: нагрузочное тестирование HTTP и gRPC сервисов доступно из одного бинарного файла без зависимостей — как через терминал, так и через браузер с живыми метриками. В отличие от существующих решений, не требуется JVM, JS-рантайм или отдельный сервер для фронтенда. Инструмент достигает заявленных нефункциональных требований: более 1 000 RPS при 50 воркерах, задержка WebSocket-обновлений 50 мс, дистрибуция для 6 платформ.